

Pokhara University
Gandaki College of Engineering and Science
Lamachaur, Kaski



Lab 5: Implementation of CI/CD Pipelines Using GitHub Actions and Jenkins

Submitted By:

Benju Pokhrel

Roll no. 18

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE:

Implementation of CI/CD Pipelines Using GitHub Actions and Jenkins

AIM:

To implement Continuous Integration and Continuous Deployment (CI/CD) pipelines using GitHub Actions and Jenkins, automating build, test, and deployment workflows for a software project.

THEORY:

CI/CD automates integration of code changes and their deployment, enabling faster and more reliable software delivery — a core Agile principle.

- **Continuous Integration (CI):** Automatically builds and tests code on every push or pull request, catching errors early.
- **Continuous Deployment (CD):** Automatically deploys tested code to a target environment after CI passes.
- **GitHub Actions:** CI/CD built into GitHub. Pipelines are defined as YAML files inside `.github/workflows/` and triggered by events like `push` or `pull_request`.
- **Jenkins:** An open-source automation server with pipelines defined in a `Jenkinsfile` using Groovy DSL. Highly extensible with hundreds of plugins.

OBJECTIVES:

- Create a GitHub Actions workflow to build and test a Node.js project on every push.
- Configure a Jenkins pipeline with build and test stages.
- Trigger pipelines and observe automated execution.
- Compare GitHub Actions and Jenkins in terms of setup and use.

ALGORITHM:

1. Create a Node.js project with a test script (Jest).
2. Push the project to a GitHub repository.
3. Create `.github/workflows/ci.yml` defining the GitHub Actions pipeline.
4. Push the workflow file and observe the Actions tab on GitHub.
5. Install Jenkins locally or on a server.
6. Create a new Jenkins Pipeline job.
7. Write a `Jenkinsfile` in the project root.
8. Trigger the Jenkins build and review the console output.

CODE:

GitHub Actions – `.github/workflows/ci.yml`:

```
name: CI Pipeline
on:
```

```

push:
  branches: [ main ]
pull_request:
  branches: [ main ]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '18'
      - run: npm install
      - run: npm test

```

Jenkins – Jenkinsfile (Declarative Pipeline):

```

pipeline {
  agent any
  stages {
    stage('Checkout') { steps { git 'https://github.com/user/repo.git' } }
    stage('Install') { steps { sh 'npm install' } }
    stage('Test') { steps { sh 'npm test' } }
    stage('Build') { steps { sh 'npm run build' } }
  }
  post {
    success { echo 'Pipeline completed successfully!' }
    failure { echo 'Pipeline failed.' }
  }
}

```

OUTPUT:

GitHub Actions (Actions tab):

```

✓ Checkout code
✓ Setup Node.js
✓ Install dependencies
✓ Run tests
Workflow completed successfully.

```

Jenkins console:

```

[Pipeline] stage: Install
[Pipeline] stage: Test -> 2 tests passed
[Pipeline] stage: Build
Finished: SUCCESS

```

RESULT:

CI/CD pipelines were successfully implemented using both GitHub Actions and Jenkins. Automated build and test stages executed correctly on code push.

CONCLUSION:

GitHub Actions offers seamless zero-infrastructure setup tightly integrated with GitHub, while Jenkins provides greater flexibility for complex enterprise workflows. Both are widely used in the industry and complement each other depending on project needs.